

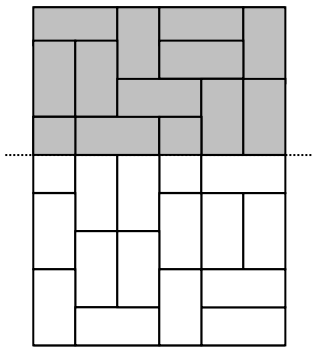
《走道铺砖问题》 解题报告

问题分析

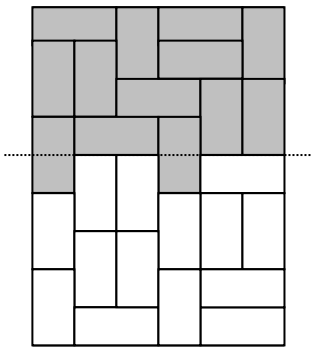
此题实际上就是一个用 1×2 的矩形覆盖 $N \times M$ 的棋盘的问题。最初用于作搜索的练习，但此题就不能光用搜索了。搜索的处理范围，大概只能到 8×8 ，而题目给出的范围却远远的超出了 8×8 。因此，必须考虑采用其它方法。

解题思路

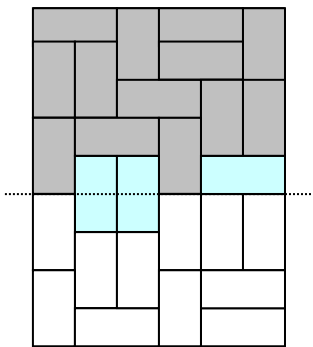
处理数据量较大的题目，很容易便想到采用动态规划的方法，此题也不例外。在对此题进行动态规划的难点在于：如何划分阶段。如果阶段划分不好，就会导致状态描述过于复杂，使空间难以承受。从最普通的开始，用横线来划分阶段(如图一)，虽然划分后包含了某些砖，但却把其中一些砖分成了两部分。从表面上看去似乎毫无规律，但是，如果将分成两部分的砖全部包含进来(如图二)，随显得有些参差不齐，但高低最多也只差一格。如果再将横线下移(如图三)，比起图二来，也就多出了几块砖而已，而多出的砖又正好填满了图二中凹的部分。如果枚举将图二中凹的部分填满的方法，那么所有填补方法所得出的新状态，它们的方案总数都应加上图二的方案总数。



图一



图二



图三

下面便使具体的状态表示方法了。对于横线 i 划出的图形，虽参差不齐，但只有凸和凹

两种，若该排有 n 个格子，就会有 2^n 种不同状态，若将凸的部分看作 1，凹的部分看作 0，再将此 n 位的二进制数转化为十进制数 j ，用 $a[i, j]$ 便可表示该状态所对应的方案总数。递推关系式为： $a[i, j] = \sum a[i-1, k]$ (k 的条件为：将其对应状态中凹的部分填满可成为状态 (i, j))。初始条件为 $a[0, 2^n-1]=1$ ，而最后的 $a[\max\{N, M\}, 0]$ 便是方案的总数。在空间方面，由于 $\min\{N, M\} \leq 12$ ，所以 $2^n \leq 4096$ ，用两个包含 2^n 个元素的数组，交换使用即可。

想出了动态规划的方法后，是否此题就顺利解决了呢？通过试验便知，还需使用高精度，以及 1 个字节记两位的压缩方法，此题方可拿到满分。

程序

```
{M 4096, 0, 655360}
const st1='input.txt';      {输入文件名}
      st2='output.txt';     {输出文件名}
      p:array[1..13] of word=(1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096);
      {p[i]代表二进制的第 i 位为 1}
type arr=array[0..15] of word;      {最多 60 位的高精度数(一个 word 型记 4 位)}
var a,b:array[0..4095] of ^arr;      {a 表示上阶段, b 表示当前阶段}
    m,n:integer;
    t:string;

procedure readp;      {读入数据}
var f:text;
begin
    assign(f,st1);reset(f);
    readln(f,n,m);
    close(f);
end;

procedure main;      {动态规划求解}
var i,j,now:word;
    f:text;

    procedure jia(var a,b:arr);      {将 a,b 两个高精度数相加后记为 a}
    var i:integer;
        o:longint;
    begin
        if b[0]>a[0] then a[0]:=b[0];
        o:=0;
        for i:=1 to a[0] do begin
            inc(o,a[i]+b[i]);
            a[i]:=o mod 10000;
            o:=o div 10000;
```

```

        end;
        if o<>0 then begin
            inc(a[0]);a[a[0]]:=o
        end;
    end;

    procedure try(j,o:integer);      {搜索状态 j 的第 o 列即第 o 个二进制位}
    begin
        while j and p[o]=p[o] do inc(o);    {找到一个凹位 o}
        if o=m+1 then begin
            jia(b[now]^,a[j]^);exit {找到了一种状态，新状态的方案总数增加 a[j]}
        end;
        now:=now+p[o];                    {将方块竖放}
        try(o+1);
        now:=now-p[o];
        if (o<m) and (j and p[o+1]=0) then try(o+2);
        {判断该凹位的右边是否也是凹位，是则可以横放}
    end;

begin
    if n<m then begin          {使 n>m}
        i:=m;m:=n;n:=i;
    end;
    for i:=0 to p[m+1]-1 do begin
        new(a[i]);fillchar(a[i]^,sizeof(arr),0);
        new(b[i]);fillchar(b[i]^,sizeof(arr),0);
    end;
    a[0]^[0]:=1;a[0]^[1]:=1;
    for i:=1 to n do begin      {n 个阶段}
        for j:=0 to p[m+1]-1 do {每阶段 2^m 种状态}
            if a[j]^[0]<>0 then begin
                now:=0;
                try(j,1);      {搜索状态 j 可以推出的所有新状态，1 表示从第一列开始}
            end;
        for j:=0 to p[m+1]-1 do begin {将新阶段变为上阶段，为求下一阶段做准备}
            a[j]^:=b[j]^;
            fillchar(b[j]^,sizeof(arr),0);
        end;
    end;
    assign(f,st2);rewrite(f);
    write(f,a[0]^[a[0]^[0]]);      {首位不需补 0，直接输出}
    for i:=a[0]^[0]-1 downto 1 do begin
        str(a[0]^[i],t);
        while length(t)<4 do t:='0'+t;      {对位数不足的补 0}
    end;
end;

```

```
        write(f,t);
    end;
    writeln(f);
    close(f);      {将结果输出到文件}
end;

begin
    readp;      {读入}
    main;      {求解并输出}
end.
```